COSC204
Computer Systems

Combinatorial Logic

Overview

- Last Lecture
  - Introduction
  - Bits and Bytes
- This Lecture
  - Combinatorial Logic
  - Source: Chapter 12 (11th edition)
  - Source: J.R. Gregg, Ones and Zeros
- Next Lecture
  - Sequential Logic

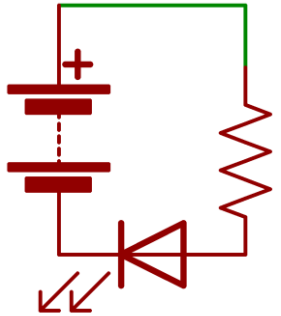
Combinatorial Logic Circuit

- A circuit whose digital outputs are dependent only on its digital inputs
- Can be described using logic expressions
  - And therefore, logic gates
- Assume the outputs respond immediately
  - In real circuits, propagation delays must be considered
    - Hence the clock-cycle on your PC

Revision

- By digital we mean binary
  - True = High = On = 1
  - False = Low = Off = 0

- We assume you have a basic understanding of a circuit
  - Current flows from + to –
  - Input to a unit (e.g. LED) is the + end
  - Output is the – end



- We assume you can do basic arithmetic in binary
  - A + B in base 2

Logic Gates

- 1 input → 1 output
  - NOT (Invert)

- 2 input → 1 output
  - AND
  - OR
  - NAND
  - NOR
  - XOR, EOR (Exclusive OR)

NOT (Invert)

$$Q = \bar{A}$$



A	Q
0	1
1	0

- We're going to use **Q** as the output so that it doesn't look like a 0 (zero)

How Many Functions

- How many functions are there of 1 binary input with 1 binary output?

A	Q
0	0
1	0

FALSE

A	Q
0	0
1	1

IDENTITY

A	Q
0	1
1	0

NOT

A	Q
0	1
1	1

TRUE

How Many Functions

- How many functions are there of 2 binary inputs with 1 binary output?
- Which ones are useful?

A	B	Q
0	0	0
0	1	0
1	0	1
1	1	1

Also written

	B=0	B=1
A=0	0	0
A=1	1	1

What is the name of this function? Is it useful?

AND

$\mathbf{Q = A \cdot B}$

- Can be extended: $\mathbf{Q = A \cdot B \cdot C \cdot \ldots}$
- "Bread **and** butter"



A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

NAND (AND then NOT)

$$Q = \overline{A \cdot B}$$



A	B	Q
0	0	1
0	1	1
1	0	1
1	1	0

OR

$\mathbf{Q = A + B}$

- Can be extended: $\mathbf{Q = A + B + C + \ldots}$
- Bring “a pen **or** a pencil” to the test



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1

NOR (OR then NOT)

$$\mathbf{Q = \overline{A + B}}$$



A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

XOR / EOR (exclusive OR)

$$Q = A \oplus B = \bar{A} \bullet B + A \bullet \bar{B}$$

- You may take either “the bus **or** the plane”



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

- Exclusive or because it excludes both

Combinatorial Logic Circuit

- A combinatorial logic circuit is a circuit whose digital outputs are dependent only on its digital inputs



- We've already seen examples: NOT, AND, OR, etc.
- Now we extend to more than 2 inputs

Define a Combinatorial Circuit

- Using a truth table
  - For each of the 2^n possible combinations of input signals, define the binary value of each of the *m* output signals

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1

Define a Combinatorial Circuit

- Using Boolean equations
  - Each output signal is expressed as a Boolean function of its input signals

$$\mathbf{Q = A + B}$$

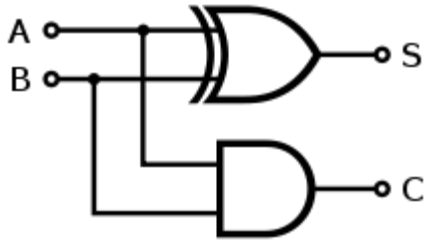
Define a Combinatorial Circuit

- Using graphical symbols
  - Draw the gates and connections used to define the circuit



2 Inputs → 2 Outputs

- What about 2 inputs → 2 outputs
  - What does this describe?



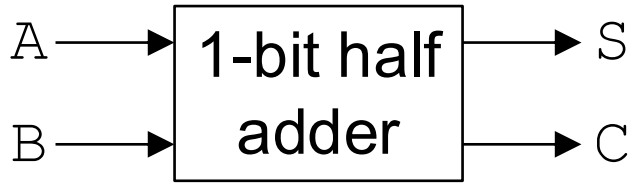
$$S = A \oplus B$$
$$C = A \bullet B$$

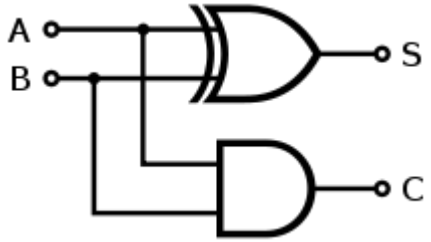
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

1-Bit Half Adder

- **S** is the sum, **C** is the carry for 1-bit arithmetic: **A+B**

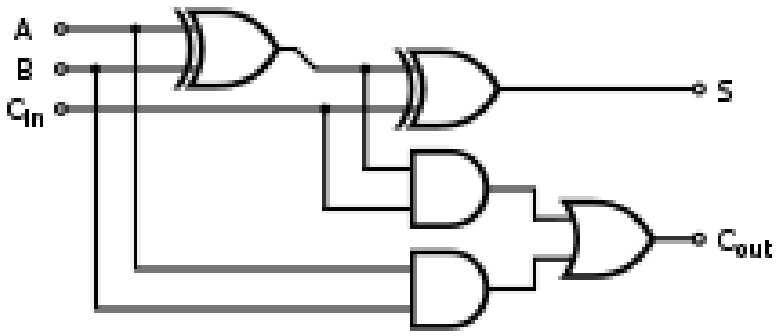
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0





1-Bit Full Adder

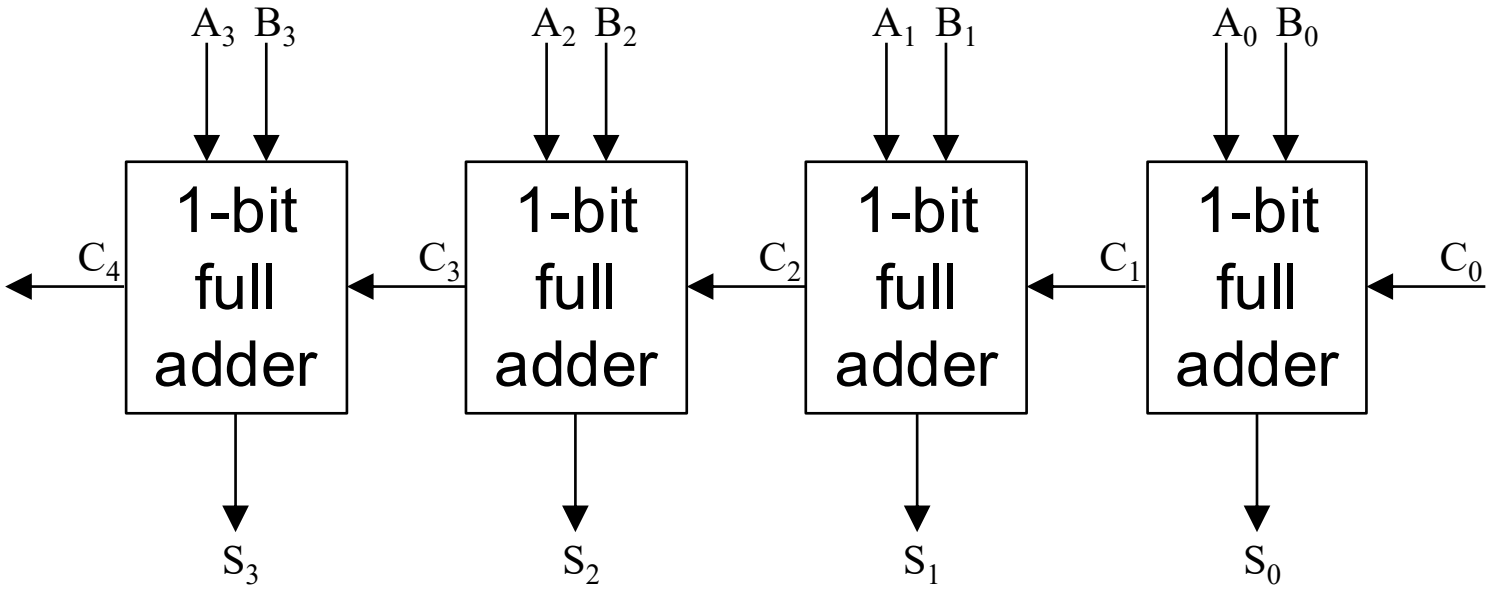
- But the half adder doesn't include ***input*** for the carry
  - The full adder does
  - And output for its carry



A	B	C_{in}	C_{out}	S
0	0	0	0	0
1	0	0	0	1
0	1	0	0	1
1	1	0	1	0
0	0	1	0	1
1	0	1	1	0
0	1	1	1	0
1	1	1	1	1

Ripple Carry Adder

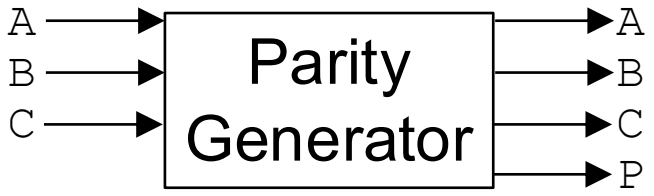
- With full adders we can define a circuit that does plus to an arbitrary number of binary digits



- C_0 is normally 0, but we'll see later that in a computer this is not always the case

What Else Can We Do?

- 3-Bit Parity Generator:
  - Add an extra bit to the input data such that the number of ones in the output data is always odd
  - Three inputs (`A`, `B`, `C`)
  - One additional output (`P`)
  - Used in signal transmission, RAID arrays, CPU caches, etc.



3-Bit Parity Generator

- Draw the truth table for `P` given `A`, `B`, `C`
  - `A`, `B`, `C` in the output are `A`, `B`, `C` from the input
    - So we only need to define `P`

#	**A**	**B**	**C**	**P**
0	0	0	0	1
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

3-Bit Parity Generator

- Write the Boolean equation
  - In ‘sum of products form’

$$P = \overline{A}\bullet\overline{B}\bullet\overline{C} + \overline{A}\bullet B\bullet C + A\bullet\overline{B}\bullet C + A\bullet B\bullet\overline{C}$$

#	**A**	**B**	**C**	**P**
0	0	0	0	1
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

3-Bit Parity Generator

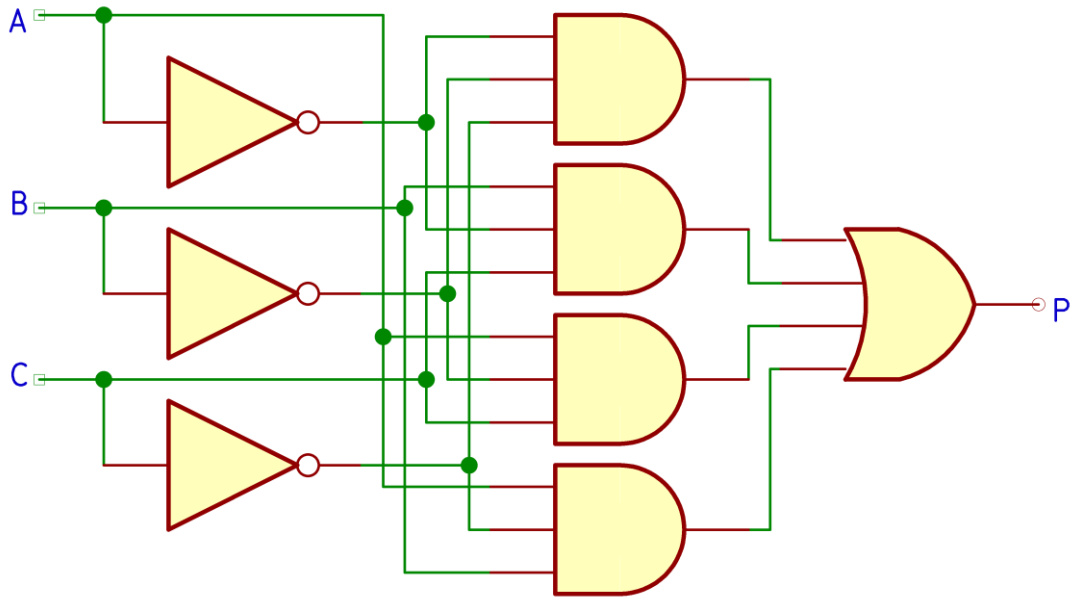
- How did we do that?
  - Look for a row where `P=1`
  - Write the equation as a function of the inputs (using AND)
  - Write the final equation putting an OR between each clause

#	A	B	C	P	
0	0	0	0	1	$\overline{A}\bullet\overline{B}\bullet\overline{C}$
1	0	0	1	0	
2	0	1	0	0	
3	0	1	1	1	$\overline{A}\bullet B\bullet C$
4	1	0	0	0	
5	1	0	1	1	$A\bullet\overline{B}\bullet C$
6	1	1	0	1	$A\bullet B\bullet\overline{C}$
7	1	1	1	0	

$$P = \overline{A}\bullet\overline{B}\bullet\overline{C} + \overline{A}\bullet B\bullet C + A\bullet\overline{B}\bullet C + A\bullet B\bullet\overline{C}$$

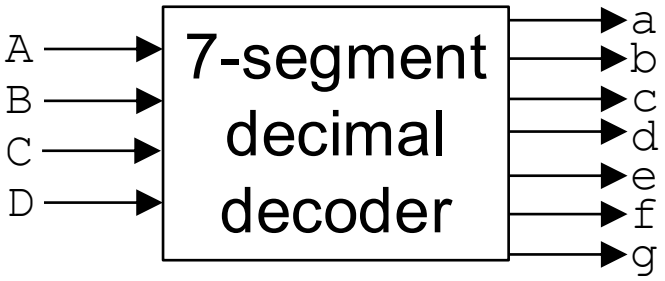
3-Bit Parity Generator

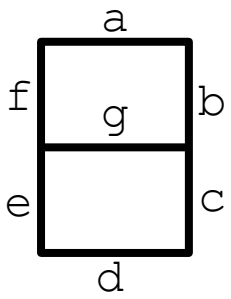
- From the equations we can define the circuit



7-Segment Digit Decoder

- The 7-segment LED display for a digital clock





Value	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	0	0	1	1

Time For a Little Bit of Maths

(Sorry)

Boolean Operator Precedence

- NOT is unary, so it has the highest precedence
- AND is like multiply, so it comes next
- OR is like plus, so it comes last

Operator	Precedence
- (NOT)	1
• (AND)	2
+ (OR)	3

De Morgan's Theorum

- De Morgan's Theorem states

$$\overline{\mathbf{A} + \mathbf{B}} = \bar{\mathbf{A}} \bullet \bar{\mathbf{B}}$$

$$\overline{\mathbf{A} \bullet \mathbf{B}} = \bar{\mathbf{A}} + \bar{\mathbf{B}}$$

- The theorem holds for any number of inputs

$$\overline{\mathbf{A} \bullet \mathbf{B} \bullet \mathbf{C} \ldots \bullet \mathbf{X}} = \bar{\mathbf{A}} + \bar{\mathbf{B}} + \bar{\mathbf{C}} + \ldots + \bar{\mathbf{X}}$$

Proof by Truth Table

- First version of De Morgan's theorem ($\overline{\mathbf{A + B}} = \bar{\mathbf{A}} \bullet \bar{\mathbf{B}}$)

$\mathbf{A}$	$\mathbf{B}$	$\mathbf{A + B}$	$\overline{\mathbf{A + B}}$	$\bar{\mathbf{A}}$	$\bar{\mathbf{B}}$	$\bar{\mathbf{A}} \bullet \bar{\mathbf{B}}$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

De Morgan

- Any Boolean function can be represented as the logical sum of logical products
- And it's possible to convert a NOR circuit into a NAND circuit and vice versa
- And AND and OR can be described by combinations of NAND gates (or NOR gates)
- So all combinatorial circuits can be described using just one gate type (either NAND or NOR)
  - The Apollo Guidance Computer used about 5600 NOR gates and no other gate types!

NAND Logic

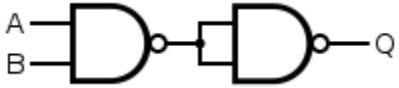
- NOT





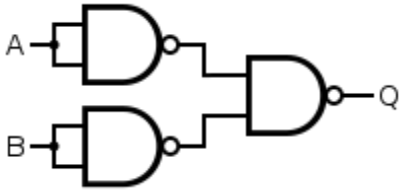
- AND





- OR





NAND Logic

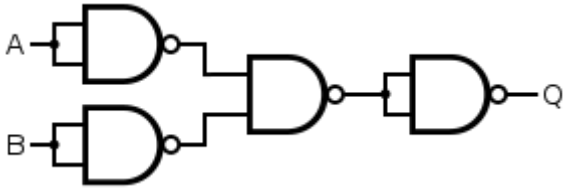
- NAND





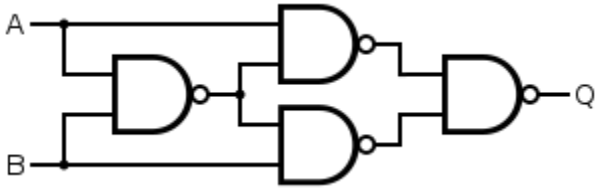
- NOR





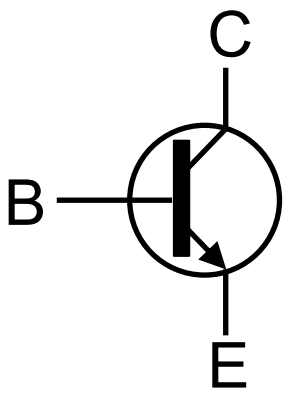
- XOR





Transistors

- The transistor is an *electronic switch*
  - B=Base, C=Collector, E=Emitter
    - Apply 0v to B and the switch opens
      - C is ***disconnected*** from E
    - Apply +5v to B and the switch closes
      - C is ***connected*** to E

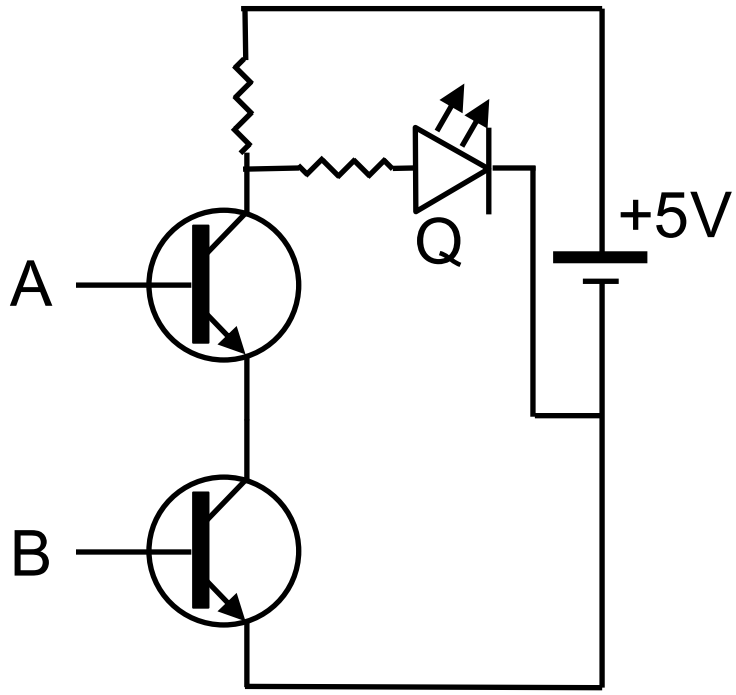


- In other words:
  - B is the switch connecting C to E

NAND Gate from Transistors

- Imagine Q is an LED, when does it shine?

- For Q to be Low (binary 0)
  - A and B must be closed
- Otherwise
  - If A is open
    - +5V flows to Q
  - If A is closed but B is open
    - +5V flows to Q
  - If A and B are closed
    - “short circuit around Q”



Transistors

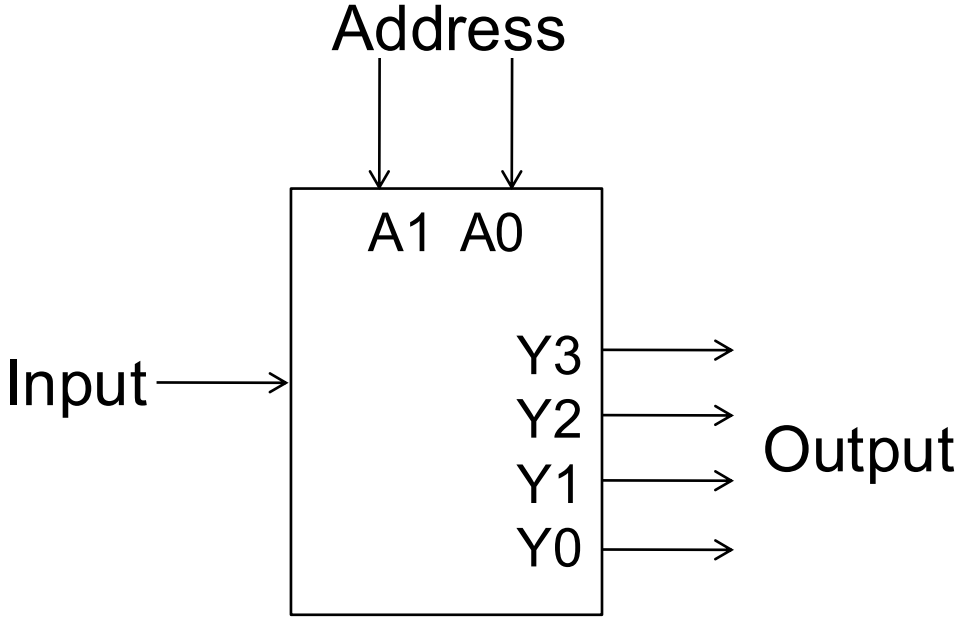
- In summary, using just transistors, it's possible to define NAND, and from NAND it's possible to define all the basic logic gates (NOT, AND, OR, etc.) and from that it's possible to define any circuit whose output is only dependant in its input (i.e. that doesn't save state)

- I think this is pretty amazing!

- But, in practice, we don't do this because it uses too many transistors. See the lab.

Demultiplexor

- A demultiplexor is a multi-way switch where the address selects which output receives the input
- In this example, a 2-bit address selects which of the 4 outputs is connected to the input



A1	A0	Output Location
0	0	Y0
0	1	Y1
1	0	Y2
1	1	Y3

Summary

- Logic gates
  - Symbols
  - Truth tables
- Combinatorial logic circuits
  - Adders (half, full, ripple-carry)
  - Parity
  - 7-Segment display decoder
  - Equations from truth tables
- Maths
- Transistors
  - Make a NAND gate

Homework

- Using only NOR gates, draw the circuit for
  - NOT, AND, OR, NAND, NOR, XOR
- Draw a half adder using only NAND gates